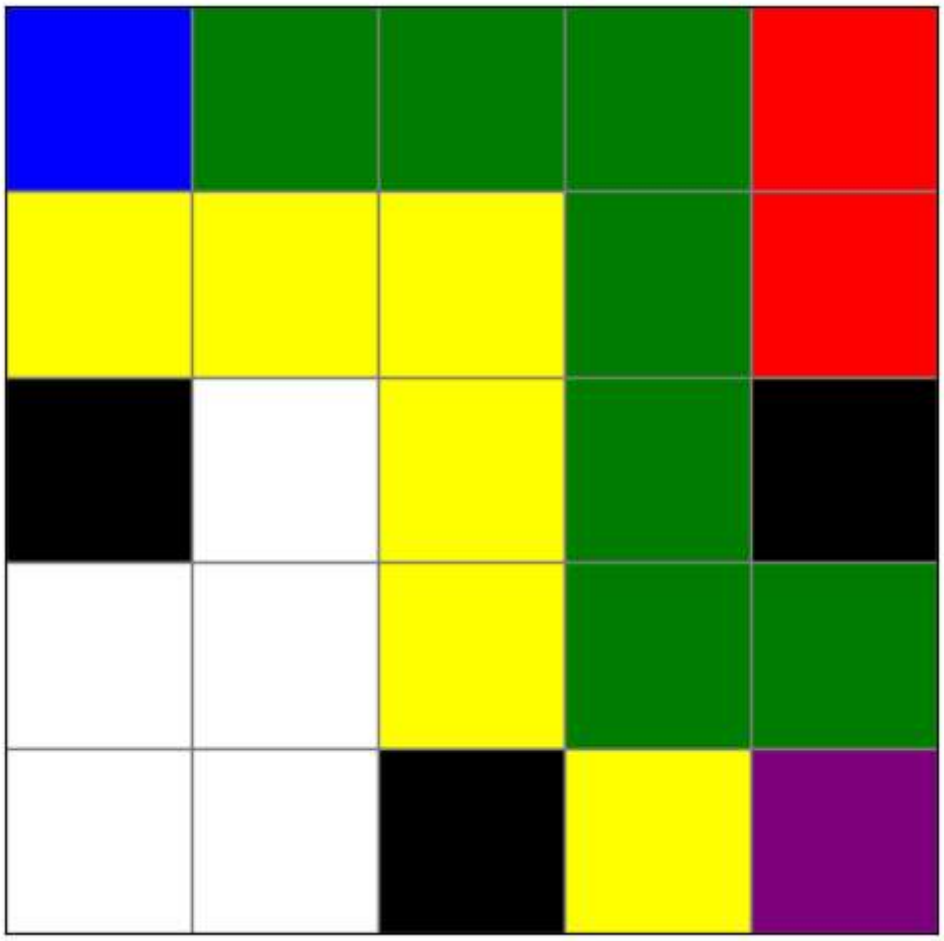
In [1]:
```
%gui tk
```

In [6]:
```
import matplotlib.pyplot as plt# for visualization
import matplotlib.patches as patches
import heapq
import random
import time
import math
from IPython.display import clear_output


ROWS = 5
OBSTACLE_DENSITY = 0.25 # means on grid obstacle

start = (0, 0)
goal = (ROWS-1, ROWS-1)

# HEURISTICS


def manhattan(a, b):#manhattan
    return abs(a[0]-b[0]) + abs(a[1]-b[1])

def euclidean(a, b):#euclidean
    return math.sqrt((a[0]-b[0])**2 + (a[1]-b[1])**2)

# Choose algorithm and heuristic here
ALGORITHM = "greedy"   # "astar" or "greedy"
HEURISTIC = manhattan # manhattan or euclidean
DYNAMIC_MODE = True

# grid cration

grid = [[0 for _ in range(ROWS)] for _ in range(ROWS)]

# Random obstacles
for i in range(ROWS):
    for j in range(ROWS):
        if random.random() < OBSTACLE_DENSITY and (i,j)!=start and (i,j)!=goal:
            grid[i][j] = 1

# DRAW FUNCTION


def draw_grid(visited=set(), frontier=set(), path=[]):
    clear_output(wait=True)
    fig, ax = plt.subplots(figsize=(6,6))

    for i in range(ROWS):
        for j in range(ROWS):
            color = "white"

            if grid[i][j] == 1:
                color = "black"
```

```
            if (i,j) in visited:
                color = "red"
            if (i,j) in frontier:
                color = "yellow"
            if (i,j) in path:
                color = "green"
            if (i,j) == start:
                color = "blue"
            if (i,j) == goal:
                color = "purple"

            rect = patches.Rectangle((j, ROWS-i-1), 1, 1,
                                     facecolor=color,
                                     edgecolor="gray")
            ax.add_patch(rect)

    ax.set_xlim(0, ROWS)
    ax.set_ylim(0, ROWS)
    ax.set_xticks([])
    ax.set_yticks([])
    plt.show()

# SEARCH FUNCTION


def search():
    open_set = []
    heapq.heappush(open_set, (0, start))

    came_from = {}
    g = {start: 0}

    visited = set()
    frontier = {start}

    start_time = time.time()

    print("\nStep | Node | g(n) | h(n) | f(n)")
    print("----------------------------------")

    step = 0

    while open_set:

        # Dynamic obstacle
        if DYNAMIC_MODE and random.random() < 0.05:
            r = random.randint(0, ROWS-1)
            c = random.randint(0, ROWS-1)
            if (r,c)!=start and (r,c)!=goal:
                grid[r][c] = 1

        f, current = heapq.heappop(open_set)
        frontier.discard(current)

        if current == goal:
            break
```

```
        visited.add(current)

        for dx, dy in [(1,0),(-1,0),(0,1),(0,-1)]:
            nx, ny = current[0]+dx, current[1]+dy

            if 0 <= nx < ROWS and 0 <= ny < ROWS:
                if grid[nx][ny] == 1:
                    continue

                temp_g = g[current] + 1

                if (nx,ny) not in g or temp_g < g[(nx,ny)]:
                    came_from[(nx,ny)] = current
                    g[(nx,ny)] = temp_g

                    h = HEURISTIC((nx,ny), goal)

                    if ALGORITHM == "astar":
                        f_value = temp_g + h
                    else:#greedy
                        f_value = h

                    heapq.heappush(open_set, (f_value, (nx,ny)))
                    frontier.add((nx,ny))

                    step += 1
                    print(f"{step:4} | {(nx,ny)} | {temp_g:4} | {round(h,2):4} | {r

        draw_grid(visited, frontier)
        time.sleep(0.1)


    # RECONSTRUCT PATH

    path = []
    node = goal
    while node in came_from:
        path.append(node)
        node = came_from[node]

    end_time = time.time()

    draw_grid(visited, frontier, path)

    print("\nFINAL PATH:", path[::-1])
    print("TOTAL COST:", len(path))
    print("NODES VISITED:", len(visited))
    print("EXECUTION TIME (ms):", round((end_time-start_time)*1000,2))


# main


draw_grid()
search()
```



```
FINAL PATH: [(0, 1), (0, 2), (0, 3), (1, 3), (2, 3), (3, 3), (3, 4), (4, 4)]
TOTAL COST: 8
NODES VISITED: 10
EXECUTION TIME (ms): 2034.83
```

In [ ]: